

SQL SERVER LOGIN FAILURE MONITORING, REPORTING & ALERTING

Production DBA Technical SOP

Independent Production Implementation

**Drafted By
DB Admin**

Contents

1. Purpose and Authorship
2. Design Principle
3. Error 18456 Fundamentals
4. Recommended Monitoring Architecture
5. Baseline Error-Log Verification
6. Primary Implementation - SQL Server Audit
7. Audit Repository
8. Independent Collection Procedure
9. Independent Aggregation and Threshold Logic
10. HTML Database Mail Alert Procedure
11. SQL Server Agent Automation
12. Extended Events Diagnostic Session
13. Error 18456 Troubleshooting Matrix
14. Production Scenarios
15. Retention and Cleanup
16. Database Mail Health Check
17. Security Best Practices
18. Validation Lab
19. Production Acceptance Checklist
20. Microsoft Technical References

1. Purpose and Authorship

This SOP provides a production-ready method to detect, retain, summarize, report, and investigate SQL Server authentication failures. The implementation in this edition is written independently and uses Microsoft SQL Server features as the technical foundation.

2. Design Principle

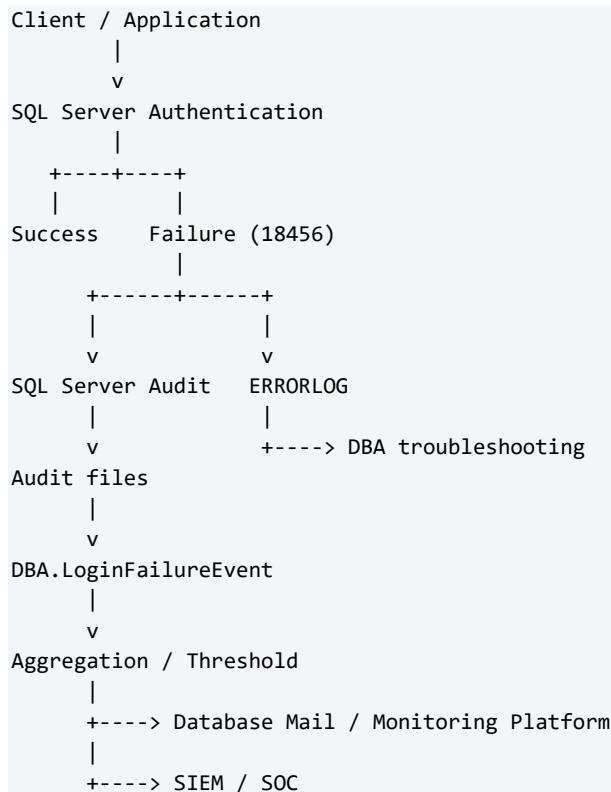
- Use SQL Server Audit as the primary structured source for failed-login evidence.
- Use the SQL Server error log for quick operational verification and Error 18456 troubleshooting.
- Use Extended Events when deeper diagnostic context is required.
- Persist normalized events in a DBA repository for history and reporting.
- Aggregate events before alerting so repeated failures do not create email storms.
- Use configurable thresholds; do not treat example thresholds as Microsoft defaults.

3. Error 18456 Fundamentals

```
Msg 18456
Login failed for user '<username>'.
```

Microsoft identifies 18456 as an authentication failure. The SQL Server error log can contain additional reason/state information that is intentionally not fully exposed to the client.

4. Recommended Monitoring Architecture



5. Baseline Error-Log Verification

For a quick operational check, use Microsoft's documented error-log reader:

```
EXEC sys.sp_readerrorlog
    @p1 = 0,
    @p2 = 1,
    @p3 = N'Login failed';
GO
```

6. Primary Implementation - SQL Server Audit

Create a dedicated audit directory at the operating-system level first and grant only the SQL Server service account and approved administrators the required access.

```
USE master;
GO

CREATE SERVER AUDIT [DBA_LoginSecurity_Audit]
TO FILE
(
    FILEPATH = N'D:\SQLAudit\LoginSecurity\',
    MAXSIZE = 512 MB,
    MAX_ROLLOVER_FILES = 30,
    RESERVE_DISK_SPACE = OFF
)
WITH
(
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);
GO

CREATE SERVER AUDIT SPECIFICATION [DBA_LoginSecurity_Spec]
FOR SERVER AUDIT [DBA_LoginSecurity_Audit]
ADD (FAILED_LOGIN_GROUP);
GO

ALTER SERVER AUDIT [DBA_LoginSecurity_Audit]
WITH (STATE = ON);
GO

ALTER SERVER AUDIT SPECIFICATION [DBA_LoginSecurity_Spec]
WITH (STATE = ON);
GO
```

For a security investigation where correlation with successful access is required, SUCCESSFUL_LOGIN_GROUP can be enabled deliberately after considering event volume.

7. Audit Repository

```
IF DB_ID(N'DBA') IS NULL
BEGIN
    CREATE DATABASE DBA;
END;
GO
```

```

USE DBA;
GO

CREATE TABLE dbo.LoginFailureEvent
(
    LoginFailureEventID bigint IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_LoginFailureEvent PRIMARY KEY,
    EventTimeUtc datetime2(7) NOT NULL,
    ServerInstance nvarchar(256) NOT NULL,
    ServerPrincipalName sysname NULL,
    ClientIp varchar(48) NULL,
    ApplicationName nvarchar(256) NULL,
    SessionServerPrincipalName sysname NULL,
    StatementText nvarchar(4000) NULL,

    /* sys.fn_get_audit_file() returns file_name as varchar(260).
       Keep this column narrow so the unique index key remains
       well below SQL Server's 1,700-byte nonclustered key limit. */
    AuditFileName varchar(260) NOT NULL,
    AuditFileOffset bigint NOT NULL,

    CollectedAtUtc datetime2(7) NOT NULL
        CONSTRAINT DF_LoginFailureEvent_CollectedAtUtc
        DEFAULT SYSUTCDATETIME(),

    CONSTRAINT UQ_LoginFailureEvent_Source
        UNIQUE NONCLUSTERED (AuditFileName, AuditFileOffset)
);
GO

CREATE INDEX IX_LoginFailureEvent_Time
ON dbo.LoginFailureEvent(EventTimeUtc DESC)
INCLUDE (ServerPrincipalName, ClientIp, ApplicationName);
GO

CREATE INDEX IX_LoginFailureEvent_SourceAnalysis
ON dbo.LoginFailureEvent
(
    ClientIp,
    ServerPrincipalName,
    EventTimeUtc DESC
)
INCLUDE (ApplicationName);
GO

```

8. Independent Collection Procedure

This collector reads structured SQL Server Audit records and deduplicates them by audit file plus file offset. It does not parse free-form ERRORLOG text.

```

USE DBA;
GO

CREATE OR ALTER PROCEDURE dbo.usp_CollectFailedLoginAudit
    @AuditFilePattern nvarchar(4000) =
        N'D:\SQLAudit\LoginSecurity\*.sqlaudit'
AS
BEGIN
    SET NOCOUNT ON;

```

```

SET XACT_ABORT ON;

;WITH SourceRows AS
(
    SELECT
        A.event_time,
        CAST(SERVERPROPERTY(N'ServerName') AS nvarchar(256)) AS ServerInstance,
        A.server_principal_name,
        CONVERT(varchar(48), A.client_ip) AS client_ip,
        A.application_name,
        A.session_server_principal_name,
        CONVERT(nvarchar(4000), A.statement) AS StatementText,
        CONVERT(varchar(260), A.file_name) AS file_name,
        A.audit_file_offset
    FROM sys.fn_get_audit_file(@AuditFilePattern, DEFAULT, DEFAULT) AS A
    WHERE A.action_id = N'LGIF'
        AND A.succeeded = 0
)
INSERT dbo.LoginFailureEvent
(
    EventTimeUtc,
    ServerInstance,
    ServerPrincipalName,
    ClientIp,
    ApplicationName,
    SessionServerPrincipalName,
    StatementText,
    AuditFileName,
    AuditFileOffset
)
SELECT
    S.event_time,
    S.ServerInstance,
    S.server_principal_name,
    S.client_ip,
    S.application_name,
    S.session_server_principal_name,
    S.StatementText,
    S.file_name,
    S.audit_file_offset
FROM SourceRows AS S
WHERE NOT EXISTS
(
    SELECT 1
    FROM dbo.LoginFailureEvent AS T
    WHERE T.AuditFileName = S.file_name
        AND T.AuditFileOffset = S.audit_file_offset
);
END;
GO

```

9. Independent Aggregation and Threshold Logic

The following procedure summarizes recent failures by login, client, and application. Thresholds are parameters so each environment can tune them.

```

USE DBA;
GO

```

```

CREATE OR ALTER PROCEDURE dbo.usp_GetLoginFailureSummary
    @WindowMinutes int = 10,
    @MinimumFailures int = 5
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @WindowStart datetime2(7) =
        DATEADD(MINUTE, -@WindowMinutes, SYSUTCDATETIME());

    SELECT
        ServerPrincipalName,
        ClientIp,
        ApplicationName,
        COUNT_BIG(*) AS FailureCount,
        MIN(EventTimeUtc) AS FirstFailureUtc,
        MAX(EventTimeUtc) AS LastFailureUtc,
        CASE
            WHEN COUNT_BIG(*) >= 100 THEN N'CRITICAL'
            WHEN COUNT_BIG(*) >= 25  THEN N'HIGH'
            WHEN COUNT_BIG(*) >= 10  THEN N'MEDIUM'
            ELSE N'LOW'
        END AS OperationalSeverity
    FROM dbo.LoginFailureEvent
    WHERE EventTimeUtc >= @WindowStart
    GROUP BY
        ServerPrincipalName,
        ClientIp,
        ApplicationName
    HAVING COUNT_BIG(*) >= @MinimumFailures
    ORDER BY FailureCount DESC;
END;
GO

```

The severity values above are local operational examples, not Microsoft-defined security severities.

10. HTML Database Mail Alert Procedure

This procedure sends a formatted HTML Database Mail alert when the configured failed-login threshold is reached. The email shows only Host Name, SQL Server Name, and Server Time Zone in the server section; it displays Alert Generated, First Failure, and Last Failure in the SQL Server host local time, while the repository continues to retain audit timestamps in UTC.

```

USE DBA;
GO

CREATE OR ALTER PROCEDURE dbo.usp_AlertOnLoginFailures
(
    @MailProfile SYSNAME
    , @Recipients NVARCHAR(MAX)
    , @WindowMinutes INT = 10
    , @MinimumFailures INT = 5
)
AS
BEGIN
    SET NOCOUNT ON;

    -----
    -- 1. Validate parameters

```

```

-----
IF @WindowMinutes <= 0
    THROW 50001, 'WindowMinutes must be greater than zero.', 1;

IF @MinimumFailures <= 0
    THROW 50002, 'MinimumFailures must be greater than zero.', 1;

-----
-- 2. Variables
-----
DECLARE
    @CurrentUtc          DATETIME2(7)
    , @WindowStartUtc    DATETIME2(7)
    , @AlertGeneratedLocal DATETIME2(7)
    , @HostName          NVARCHAR(128)
    , @SqlServerName     NVARCHAR(128)
    , @ServerTimeZone    SYSNAME
    , @Subject           NVARCHAR(255)
    , @Body              NVARCHAR(MAX)
    , @Rows              NVARCHAR(MAX);

-----
-- 3. Time values
-- Repository timestamps remain UTC for audit consistency.
-- Email timestamps are shown in SQL Server host local time.
-----
SET @CurrentUtc = SYSUTCDATETIME();

SET @WindowStartUtc =
    DATEADD(MINUTE, -@WindowMinutes, @CurrentUtc);

SET @AlertGeneratedLocal =
    SYSDATETIME();

-----
-- 4. Server details
-----
SET @HostName =
    CONVERT(NVARCHAR(128), SERVERPROPERTY(N'MachineName'));

SET @SqlServerName =
    CONVERT(NVARCHAR(128), SERVERPROPERTY(N'ServerName'));

-----
-- 5. Get Windows server time zone
-----
BEGIN TRY
    EXEC master.dbo.xp_regread
        N'HKEY_LOCAL_MACHINE',
        N'SYSTEM\CurrentControlSet\Control\TimeZoneInformation',
        N'TimeZoneKeyName',
        @ServerTimeZone OUTPUT;
END TRY
BEGIN CATCH
    SET @ServerTimeZone = NULL;
END CATCH;

IF @ServerTimeZone IS NULL
    SET @ServerTimeZone = N'Server Local Time';

-----
-- 6. Aggregate login failures
-----
DECLARE @Summary TABLE

```

```

(
    LoginName          SYSNAME
    , ClientIp          VARCHAR(48)
    , ApplicationName    NVARCHAR(256)
    , FailureCount       BIGINT
    , FirstFailureLocal  NVARCHAR(50)
    , LastFailureLocal   NVARCHAR(50)
    , Severity           VARCHAR(20)
);

INSERT INTO @Summary
(
    LoginName
    , ClientIp
    , ApplicationName
    , FailureCount
    , FirstFailureLocal
    , LastFailureLocal
    , Severity
)
SELECT
    ServerPrincipalName
    , ClientIp
    , ApplicationName
    , COUNT_BIG(*)
    , CASE
        WHEN @ServerTimeZone <> N'Server Local Time'
        THEN CONVERT
            (
                NVARCHAR(50),
                (
                    MIN(EventTimeUtc)
                    AT TIME ZONE 'UTC'
                    AT TIME ZONE @ServerTimeZone
                ),
                121
            )
        ELSE CONVERT
            (
                NVARCHAR(50),
                DATEADD
                (
                    SECOND,
                    DATEDIFF
                    (
                        SECOND,
                        SYSUTCDATETIME(),
                        SYSDATETIME()
                    ),
                    MIN(EventTimeUtc)
                ),
                121
            )
        END
    , CASE
        WHEN @ServerTimeZone <> N'Server Local Time'
        THEN CONVERT
            (
                NVARCHAR(50),
                (
                    MAX(EventTimeUtc)
                    AT TIME ZONE 'UTC'
                    AT TIME ZONE @ServerTimeZone
                ),
            )
    ,

```

```

121
    )
ELSE CONVERT
(
    NVARCHAR(50),
    DATEADD
    (
        SECOND,
        DATEDIFF
        (
            SECOND,
            SYSUTCDATETIME(),
            SYSDATETIME()
        ),
        MAX(EventTimeUtc)
    ),
    121
)
END
, CASE
    WHEN COUNT_BIG(*) >= 100 THEN 'CRITICAL'
    WHEN COUNT_BIG(*) >= 25 THEN 'HIGH'
    WHEN COUNT_BIG(*) >= 10 THEN 'MEDIUM'
    ELSE 'WARNING'
END
FROM dbo.LoginFailureEvent
WHERE EventTimeUtc >= @WindowStartUtc
GROUP BY
    ServerPrincipalName
    , ClientIp
    , ApplicationName
HAVING COUNT_BIG(*) >= @MinimumFailures;

-----
-- 7. No qualifying failures = no email
-----
IF NOT EXISTS (SELECT 1 FROM @Summary)
    RETURN;

-----
-- 8. Build HTML login failure rows
-----
SELECT @Rows =
(
    SELECT
        '<tr>' +
        '<td style="border:1px solid #BFBFBF;padding:8px;">' +
        COALESCE(
            REPLACE(REPLACE(REPLACE(LoginName,
                '&', '&');', '<', '<');', '>', '>');',
            '<unknown>';
        ) +
        '</td>' +
        '<td style="border:1px solid #BFBFBF;padding:8px;">' +
        COALESCE(
            REPLACE(REPLACE(REPLACE(ClientIp,
                '&', '&');', '<', '<');', '>', '>');',
            '<unknown>';
        ) +
        '</td>' +
        '<td style="border:1px solid #BFBFBF;padding:8px;">' +
        COALESCE(
            REPLACE(REPLACE(REPLACE(ApplicationName,
                '&', '&');', '<', '<');', '>', '>');',

```

```

        '&lt;unknown&gt;';
    ) +
    '</td>' +
    '<td style="border:1px solid #BFBFBF;padding:8px;text-align:center;font-weight:bold;">' +
    CONVERT(VARCHAR(20), FailureCount) +
    '</td>' +
    '<td style="border:1px solid #BFBFBF;padding:8px;white-space:nowrap;">' +
    FirstFailureLocal +
    '</td>' +
    '<td style="border:1px solid #BFBFBF;padding:8px;white-space:nowrap;">' +
    LastFailureLocal +
    '</td>' +
    '<td style="border:1px solid #BFBFBF;padding:8px;text-align:center;font-weight:bold;">' +
    Severity +
    '</td>' +
    '</tr>'
FROM @Summary
ORDER BY FailureCount DESC
FOR XML PATH(''), TYPE
).value('.', 'NVARCHAR(MAX)');

-----
-- 9. Email subject
-----
SET @Subject =
    N'SQL Server Login Failure Alert - ' + @SqlServerName;

-----
-- 10. Build HTML email
-----
SET @Body =
N'<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
</head>
<body style="font-family:Segoe UI,Arial,sans-serif;font-size:13px;color:#222222;">

<h2 style="color:#C00000;margin-bottom:6px;">
SQL Server Login Failure Alert
</h2>

<p>
SQL Server authentication failures have reached the configured monitoring threshold.
</p>

<h3 style="color:#1F4E79;margin-top:20px;margin-bottom:8px;">
Server Information
</h3>

<table style="border-collapse:collapse;width:700px;">
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;width:200px;font-weight:bold;background:#D9EAF7;">
Host Name
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">
+ COALESCE(@HostName,N'Unknown') +
N'</td>
</tr>
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;font-weight:bold;background:#D9EAF7;">
SQL Server Name
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">

```

```

+ COALESCE(@SqlServerName,N'Unknown') +
N'</td>
</tr>
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;font-weight:bold;background:#D9EAF7;">
Server Time Zone
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">'
+ COALESCE(@ServerTimeZone,N'Server Local Time') +
N'</td>
</tr>
</table>

<h3 style="color:#1F4E79;margin-top:20px;margin-bottom:8px;">
Monitoring Information
</h3>

<table style="border-collapse:collapse;width:700px;">
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;width:200px;font-weight:bold;background:#FFF2CC;">
Alert Generated
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">'
+ CONVERT(VARCHAR(23), @AlertGeneratedLocal, 121) +
N'</td>
</tr>
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;font-weight:bold;background:#FFF2CC;">
Monitoring Window
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">'
+ CONVERT(VARCHAR(10), @WindowMinutes) +
N' minute(s)</td>
</tr>
<tr>
<td style="border:1px solid #BFBFBF;padding:8px;font-weight:bold;background:#FFF2CC;">
Minimum Failure Threshold
</td>
<td style="border:1px solid #BFBFBF;padding:8px;">'
+ CONVERT(VARCHAR(10), @MinimumFailures) +
N' failure(s)</td>
</tr>
</table>

<h3 style="color:#1F4E79;margin-top:22px;margin-bottom:8px;">
Login Failure Audit Details
</h3>

<table style="border-collapse:collapse;width:100%;max-width:1100px;">
<thead>
<tr style="background:#1F4E79;color:#FFFFFF;">
<th style="border:1px solid #BFBFBF;padding:9px;text-align:left;">Login Name</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:left;">Client IP / Host</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:left;">Application Name</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:center;">Failure Count</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:left;">First Failure - Server Local Time</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:left;">Last Failure - Server Local Time</th>
<th style="border:1px solid #BFBFBF;padding:9px;text-align:center;">Severity</th>
</tr>
</thead>
<tbody>'
+ @Rows +
N'</tbody>
</table>

```

```

<br>

<div style="margin-top:16px;padding:12px;border-left:5px solid #C00000;background:#FCE4D6;">
<b>Recommended DBA Action</b>
<br><br>
Review the login name, originating client, application, failure frequency, and failure timing.
<ul>
<li>Incorrect or expired password</li>
<li>Application credential or secret change</li>
<li>Disabled SQL Server login</li>
<li>Stale SQL Agent job credentials</li>
<li>Incorrect application connection string</li>
<li>Unavailable default database</li>
<li>Unauthorized authentication attempt</li>
<li>Credential spraying or brute-force activity</li>
</ul>
</div>

<br>

<p style="font-size:11px;color:#666666;">
Automated SQL Server Security Monitoring Notification
<br>SQL Server: '
+ COALESCE(@SqlServerName,N'Unknown') +
N'<br>Host: '
+ COALESCE(@HostName,N'Unknown') +
N'<br>Generated: '
+ CONVERT(VARCHAR(23), @AlertGeneratedLocal, 121) +
N' - '
+ COALESCE(@ServerTimeZone,N'Server Local Time') +
N'</p>

</body>
</html>';

-----
-- 11. Send HTML email
-----
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = @MailProfile
    , @recipients = @Recipients
    , @subject = @Subject
    , @body = @Body
    , @body_format = 'HTML';

END;
GO

```

11. SQL Server Agent Automation

Create one SQL Server Agent job with separate steps so collection and alerting can be troubleshot independently.

```

-- Job Step 1
EXEC DBA.dbo.usp_CollectFailedLoginAudit;
GO

-- Job Step 2
EXEC DBA.dbo.usp_AlertOnLoginFailures
    @MailProfile = N'DBA_Alerts',
    @Recipients = N'dba-team@example.com',

```

```

        @WindowMinutes = 10,
        @MinimumFailures = 5;
GO

```

- Suggested starting schedule: every 5 minutes.
- Configure job-failure notification separately from login-failure notification.
- Keep the alert window larger than or equal to the job interval to avoid blind spots.
- Use deduplication in the repository so overlapping collection windows do not duplicate events.

12. Extended Events Diagnostic Session

Use Extended Events as a diagnostic supplement when you need event-level context around Error 18456.

```

IF EXISTS
(
    SELECT 1
    FROM sys.server_event_sessions
    WHERE name = N'DBA_LoginFailure_Diagnostics'
)
DROP EVENT SESSION [DBA_LoginFailure_Diagnostics] ON SERVER;
GO

CREATE EVENT SESSION [DBA_LoginFailure_Diagnostics]
ON SERVER
ADD EVENT sqlserver.error_reported
(
    ACTION
    (
        sqlserver.client_app_name,
        sqlserver.client_hostname,
        sqlserver.client_pid,
        sqlserver.server_principal_name,
        sqlserver.session_id
    )
    WHERE
    (
        [error_number] = 18456
        AND [severity] >= 14
    )
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\XEvents\DBA_LoginFailure_Diagnostics.xel',
    max_file_size = 100,
    max_rollover_files = 10
);
GO

ALTER EVENT SESSION [DBA_LoginFailure_Diagnostics]
ON SERVER STATE = START;
GO

```

13. Error 18456 Troubleshooting Matrix

State	Common interpretation / DBA direction
2 / 5	Login name is not valid or cannot be resolved in the expected context.
6	Windows identity used in a SQL-authentication style connection.

7	Login disabled or credential-related failure; inspect the detailed error-log reason.
8	Password mismatch.
11 / 12	Login is valid but server access is denied in the applicable context.
18	Password change is required.
38	Requested/default database cannot be opened.

State interpretation must be validated against the detailed SQL Server error-log message and current Microsoft documentation for the deployed version.

14. Production Scenarios

Application secret rotation failed

- A service account password is rotated but one application node keeps the old secret.
- Audit history shows one login/client/application combination producing repeated failures.
- Update the secret on the affected node and verify the event rate returns to baseline.

Disabled legacy login

- A retired login continues to receive connection attempts.
- Do not enable the login automatically.
- Find the originating service, job, script, or host and remove the stale dependency.

Unavailable default database

- Authentication reaches SQL Server but the requested/default database is unavailable.
- Check database state, login default database, connection string, and user mapping.
- Correct the root cause and retest.

Possible credential attack

- Many login names or a privileged login are repeatedly attempted from an unexpected source.
- Preserve Audit/XE/network evidence and correlate with successful-login activity.
- Escalate through the organization's security incident process.

15. Retention and Cleanup

```
USE DBA;
GO

CREATE OR ALTER PROCEDURE dbo.usp_PurgeLoginFailureHistory
    @RetentionDays int = 90
AS
BEGIN
    SET NOCOUNT ON;

    IF @RetentionDays < 1
        THROW 50001, 'RetentionDays must be at least 1.', 1;

    DELETE TOP (10000)
    FROM dbo.LoginFailureEvent
    WHERE EventTimeUtc < DATEADD(DAY, -@RetentionDays, SYSUTCDATETIME());
```

```

WHILE @@ROWCOUNT = 10000
BEGIN
    DELETE TOP (10000)
    FROM dbo.LoginFailureEvent
    WHERE EventTimeUtc < DATEADD(DAY, -@RetentionDays, SYSUTCDATETIME());
END
END;
GO

```

Set retention according to security, legal, and organizational requirements. Ninety days is only an example parameter value.

16. Database Mail Health Check

```

EXEC msdb.dbo.sysmail_help_status_sp;
GO

SELECT TOP (50)
    mailitem_id,
    recipients,
    subject,
    send_request_date,
    last_mod_date
FROM msdb.dbo.sysmail_faileditems
ORDER BY send_request_date DESC;
GO

SELECT TOP (50)
    log_date,
    event_type,
    description,
    process_id,
    mailitem_id
FROM msdb.dbo.sysmail_event_log
ORDER BY log_date DESC;
GO

```

17. Security Best Practices

- Use least privilege for monitoring and audit readers.
- Protect Audit and Extended Events directories with operating-system permissions.
- Do not use sa for applications.
- Coordinate password rotation with application secret updates.
- Correlate database evidence with firewall, identity, endpoint, and SIEM telemetry.
- Do not automatically block a source or disable a login solely from one threshold event without an approved response policy.
- Monitor successful authentication when investigating suspected compromise.
- Test collection and alerting after upgrades, failovers, service-account changes, or storage-path changes.

18. Validation Lab

```

USE master;
GO

```

```

CREATE LOGIN [DBA_LoginMonitor_Test]
WITH PASSWORD = N'Temporary-Lab-Only#2026',
    CHECK_POLICY = ON;
GO

-- From a test client, deliberately attempt to connect with the wrong password.
-- Then run the collector:

EXEC DBA.dbo.usp_CollectFailedLoginAudit;
GO

SELECT TOP (20)
    EventTimeUtc,
    ServerPrincipalName,
    ClientIp,
    ApplicationName
FROM DBA.dbo.LoginFailureEvent
WHERE ServerPrincipalName = N'DBA_LoginMonitor_Test'
ORDER BY EventTimeUtc DESC;
GO

DROP LOGIN [DBA_LoginMonitor_Test];
GO

```

19. Production Acceptance Checklist

- Audit directory exists and permissions are restricted.
- Server Audit and Audit Specification are enabled.
- FAILED_LOGIN_GROUP is producing events.
- DBA repository and indexes are created.
- Collector inserts new events without duplicates.
- Aggregation thresholds match the environment's baseline.
- Database Mail profile and recipients are validated.
- SQL Agent job succeeds on schedule.
- Job failure notification is configured.
- Extended Events diagnostic session is available if required.
- Retention is approved and cleanup is scheduled.
- Test failed login is captured end to end.
- Security/SOC escalation path is documented.

20. Microsoft Technical References

Drafted by: DB Admin.

- Microsoft Learn - MSSQLSERVER_18456.
- Microsoft Learn - SQL Server Audit and Server Audit Specification.
- Microsoft Learn - sys.fn_get_audit_file.
- Microsoft Learn - Database Mail / sp_send_dbmail.
- Microsoft Learn - SQL Server Agent.
- Microsoft Learn - Extended Events.

Testing:

```
-- Job Step 1
EXEC DBA.dbo.usp_CollectFailedLoginAudit;
GO
```

```
-- Job Step 2
EXEC DBA.dbo.usp_AlertOnLoginFailures
    @MailProfile = N'DBAAlerts',
    @Recipients = N'DBAdminTeam@xyz.com',
    @WindowMinutes = 1,
    @MinimumFailures = 2;
GO
```

SQL Server Login Failure Alert - PMSQLDBA > AAA.DBA_Test_Mails x SQLDBA_Alerts x

SQL Server Alerts to me 9:08 PM (12 minutes ago) ☆

SQL Server Login Failure Alert

SQL Server authentication failures have reached the configured monitoring threshold.

Server Information

Host Name	
SQL Server Name	
Server Time Zone	Central Standard Time

Monitoring Information

Alert Generated	2026-09-08 21:08:15.961
Monitoring Window	1 minute(s)
Minimum Failure Threshold	2 failure(s)

Login Failure Audit Details

Login Name	Client IP / Host	Application Name	Failure Count	First Failure - Server Local Time	Last Failure - Server Local Time	Severity
<						
DBA_LoginMonitor_Test	local machine	Microsoft SQL Server Management Studio	4	2026-09-08 21:07:55.3313029 -05:00	2026-09-08 21:08:09.4309223 -05:00	WARNING

Recommended DBA Action

Review the login name, originating client, application, failure frequency, and failure timing.

- Incorrect or expired password
- Application credential or secret change
- Disabled SQL Server login
- Stale SQL Agent job credentials
- Incorrect application connection string
- Unavailable default database
- Unauthorized authentication attempt
- Credential spraying or brute-force activity